Visualização 3D Avançada de Árvores Arteriais CCO: Visibilidade, Iluminação e Técnicas Baseadas em Raios

Brenda Gabrielle Alves Nascimento[1] (24.1.4011)

[1]Departamento de Computação (DECOM), Universidade Federal de Ouro Preto (UFOP)

BCC327 – Computação Gráfica, 2026.2 | Prof. Dr. Rafael Alves Bonfim de Queiroz

RESUMO

Contexto: Árvores arteriais sintéticas geradas pelo método *Constrained Constructive Optimization* (CCO) são estruturas 3D densas, com centenas de vasos que se cruzam em profundidade; uma imagem útil exige resolver visibilidade, dar volume por iluminação e permitir isolar e consultar vasos. **Objetivo:** Evoluir o visualizador do TP1 para uma aplicação de visualização científica 3D que implemente e compare as técnicas clássicas de visibilidade, iluminação, transparência, textura e técnicas baseadas em raios. **Metodologia:** A aplicação, em Python com pygame-ce, PyOpenGL e NumPy, desenha os vasos como tubos com normais e faces orientadas, e usa shaders GLSL próprios com o modelo de Blinn–Phong nos sombreamentos Flat, Gouraud e Phong. Foram implementados Back-Face Culling pelo critério $\mathbf{N} \cdot \mathbf{V}$, visualização linearizada do Z-Buffer, transparência com Painter's Algorithm, filtragem bilinear própria, seleção de vasos por Ray Casting (interseção raio–cilindro e raio–esfera), um mini Ray Tracer com sombras e reflexões e, como recurso adicional, métricas da árvore com filtro interativo por raio. **Resultados:** O culling próprio descarta cerca de metade dos triângulos sem alterar a imagem quando o Z-Buffer está ativo; sem Z-Buffer, nenhum dos dois tipos de culling resolve a oclusão entre vasos. As três técnicas de sombreamento, a ordenação dos transparentes e as filtragens foram comparadas visualmente; o exemplo numérico de bilinear coincide com o shader; e as métricas revelam que os 25% de vasos mais grossos concentram 57% do volume sanguíneo. **Conclusão:** O Z-Buffer é a técnica que garante a visibilidade correta; culling, ordenação e raios complementam o pipeline em desempenho, transparência e interação.

Palavras-Chave: Computação Gráfica; Método CCO; Visibilidade; Iluminação; Ray Tracing.

1 Introdução

O método CCO, introduzido por Schreiner e Buxbaum [1], gera árvores vasculares sintéticas que suprem um domínio de perfusão sob restrições fisiológicas. Cada modelo é um grafo em árvore descrito por pontos, segmentos e um raio por segmento. No TP1 esses modelos foram lidos, transformados e desenhados como linhas, faixas e tubos, e os algoritmos clássicos de rasterização e recorte foram implementados num modo didático. A iluminação dos tubos, porém, era a do pipeline fixo do OpenGL, e a visibilidade ficava inteiramente a cargo da biblioteca.

Num modelo 3D com 511 vasos, os ramos se sobrepõem em vários níveis de profundidade, de modo que a imagem só é interpretável se a oclusão estiver correta, se a iluminação revelar a forma cilíndrica de cada vaso e se o usuário puder isolar subconjuntos e consultar vasos específicos. Este trabalho implementa e compara as técnicas que tratam esses problemas. As contribuições são:

- tubos 3D com normais de vértice e de face e orientação anti-horária consistente das faces;
- Back-Face Culling próprio pelo critério $\mathbf{N} \cdot \mathbf{V}$, comparado ao do OpenGL, e visualização linearizada do Z-Buffer;
- shaders GLSL próprios com o modelo de Blinn–Phong nos sombreamentos Flat, Gouraud e Phong, com luz pontual móvel;
- transparência com Painter's Algorithm, plano de referência texturizado com filtragem bilinear própria;
- seleção de vasos por Ray Casting e um mini Ray Tracer com raios primários, de sombra e refletidos;

- como recurso adicional, métricas morfológicas da árvore e um filtro interativo por intervalo de raio.

2 Fundamentação Teórica

2.1 Modelos CCO e representação em tubos

Os modelos estão no formato VTK legado ASCII: a seção `POINTS` traz as coordenadas, em metros, `LINES` liga pares de pontos e `CELL_DATA` traz o raio de cada segmento, em milímetros. Para a renderização 3D, cada segmento de extremidades $\mathbf{a}$ e $\mathbf{b}$ e raio r é aproximado por um cilindro, cuja superfície lateral é uma malha de triângulos. O vértice k de um anel é $\mathbf{a} + r(\cos\theta_k\,\mathbf{e}_1 + \sin\theta_k\,\mathbf{e}_2)$, em que $(\mathbf{d}, \mathbf{e}_1, \mathbf{e}_2)$ é uma base ortonormal com $\mathbf{d}$ ao longo do eixo; a normal desse vértice é o próprio vetor radial $\cos\theta_k\,\mathbf{e}_1 + \sin\theta_k\,\mathbf{e}_2$.

2.2 Visibilidade

Back-Face Culling. Num objeto fechado, as faces cuja normal aponta para longe do observador estão sempre escondidas pelas faces da frente. Sendo $\mathbf{N}$ a normal da face e $\mathbf{V}$ o vetor da face para o observador, ambos no espaço da câmera, a face é descartada quando

$$\mathbf{N}\cdot\mathbf{V} \le 0, \qquad \mathbf{V} = \begin{cases} -\mathbf{c}, & \text{perspectiva (observador na origem, } \mathbf{c} = \text{centroide da face),} \\ (0,0,1), & \text{ortográfica (raios de projeção paralelos a } z). \end{cases} \tag{1}$$

O OpenGL usa um critério equivalente, aplicado depois da projeção: a face é de frente se seus vértices aparecem em sentido anti-horário na tela. Esse critério só funciona se todos os triângulos forem orientados de forma consistente, isto é, anti-horários vistos de fora, o que pela regra da mão direita equivale a $(\mathbf{v}_1 - \mathbf{v}_0) \times (\mathbf{v}_2 - \mathbf{v}_0)$ apontar para fora. O culling elimina cerca de metade dos triângulos, mas *não* resolve a oclusão entre objetos distintos: a face da frente de um vaso distante continua sendo desenhada mesmo se um vaso próximo a cobrir.

Z-Buffer. O algoritmo de Catmull [3, 2] guarda, para cada pixel, a profundidade do fragmento mais próximo já desenhado. Um novo fragmento só é escrito se estiver mais perto, de modo que o resultado independe da ordem de desenho. Na projeção perspectiva com planos n (near) e f (far), a profundidade armazenada $d \in [0,1]$ não é linear na distância z à câmera; a relação inversa é

$$z = \frac{2nf}{(f+n) - (2d-1)(f-n)}, \tag{2}$$

e a resolução do buffer se concentra perto do plano n.

2.3 Iluminação: modelo de Blinn–Phong

O modelo de reflexão de Phong [4], na variante de Blinn [5], soma três componentes num ponto de cor $\mathbf{c}$ e normal $\mathbf{N}$:

$$I = k_a\,\mathbf{c} + I_L\left[k_d\,\mathbf{c}\,\max(\mathbf{N}\cdot\mathbf{L}, 0) + k_s \max(\mathbf{N}\cdot\mathbf{H}, 0)^{\alpha}\right], \qquad \mathbf{H} = \frac{\mathbf{L}+\mathbf{V}}{\|\mathbf{L}+\mathbf{V}\|}, \tag{3}$$

em que $\mathbf{L}$ aponta para a luz pontual, $\mathbf{V}$ para o observador e I_L é a intensidade da fonte. A componente **ambiente** aproxima a luz indireta e é constante; a **difusa** (lambertiana) depende apenas do ângulo entre a normal e a luz e independe do observador; a **especular** produz o brilho, cujo tamanho diminui com o expoente α. O vetor intermediário $\mathbf{H}$ evita calcular o vetor refletido. Como as normais precisam continuar perpendiculares à superfície depois das transformações, elas são transformadas pela matriz $(M_{3\times3}^{-1})^T$, e não pela própria modelview.

2.4 Sombreamento: Flat, Gouraud e Phong

Os três métodos usam a mesma equação e diferem em *onde* ela é avaliada. No **Flat**, uma vez por triângulo, com a normal da face: a malha aparece facetada. No **Gouraud** [6], uma vez por vértice, com as normais de vértice; as cores resultantes são interpoladas no triângulo, o que suaviza a malha mas pode perder ou borrar um brilho especular que caia no interior de um triângulo grande. No **Phong** [4], as *normais* são interpoladas e a equação é avaliada em cada pixel, reproduzindo o brilho especular com fidelidade ao custo de mais operações por fragmento.

2.5 Transparência e Painter's Algorithm

Com alpha blending, a cor de um fragmento de opacidade α é composta com a já presente no framebuffer:

$$C = \alpha\, C_{\text{novo}} + (1-\alpha)\, C_{\text{antigo}}. \tag{4}$$

A operação não é comutativa: o resultado só é correto se as superfícies forem compostas de trás para frente. O Painter's Algorithm [7] ordena as primitivas pela profundidade e as desenha da mais distante à mais próxima, como um pintor que pinta o fundo primeiro. A ordenação por um único valor de profundidade (por exemplo, o centroide) falha quando triângulos se interpenetram ou se sobrepõem ciclicamente.

2.6 Mapeamento de textura e filtragem

Cada vértice recebe coordenadas $(u, v) \in [0,1]^2$ que indicam o ponto da imagem colado a ele; o rasterizador interpola (u, v) e, em cada pixel, a cor é buscada na textura. Numa textura de $n \times n$ texels, com centros em $(i + 0{,}5)/n$, a filtragem **NEAREST** usa o texel mais próximo, $i = \lfloor un \rfloor$, e produz blocos quando a textura é ampliada. A filtragem **bilinear** faz $t_x = un - 0{,}5$, $i_0 = \lfloor t_x \rfloor$, $f_x = t_x - i_0$ (e o análogo em v) e interpola os quatro vizinhos:

$$\mathbf{c} = (1-f_y)\,[(1-f_x)\,\mathbf{c}_{00} + f_x\,\mathbf{c}_{10}] + f_y\,[(1-f_x)\,\mathbf{c}_{01} + f_x\,\mathbf{c}_{11}]\,. \tag{5}$$

2.7 Ray Casting e Ray Tracing

No **Ray Casting** [8], um raio $P(t) = \mathbf{O} + t\mathbf{D}$ parte do observador por um pixel e o objeto visível é o de menor $t > 0$. Para um cilindro de eixo unitário $\mathbf{a}$, base $\mathbf{A}$ e comprimento h, com $\mathbf{m} = \mathbf{O} - \mathbf{A}$, a superfície lateral é o conjunto de pontos cuja parte perpendicular ao eixo tem comprimento r, o que leva a

$$k_2 t^2 + k_1 t + k_0 = 0, \quad k_2 = \mathbf{D}\cdot\mathbf{D} - (\mathbf{D}\cdot\mathbf{a})^2, \quad k_1 = 2\,[\mathbf{m}\cdot\mathbf{D} - (\mathbf{m}\cdot\mathbf{a})(\mathbf{D}\cdot\mathbf{a})]\,, \quad k_0 = \mathbf{m}\cdot\mathbf{m} - (\mathbf{m}\cdot\mathbf{a})^2 - r^2, \tag{6}$$

e a raiz só vale se a projeção do ponto no eixo, $s = \mathbf{m}\cdot\mathbf{a} + t\,\mathbf{D}\cdot\mathbf{a}$, estiver em $[0, h]$. Para a esfera de centro $\mathbf{C}$, com $\mathbf{m} = \mathbf{O} - \mathbf{C}$: $(\mathbf{D}\cdot\mathbf{D})t^2 + 2(\mathbf{m}\cdot\mathbf{D})t + \mathbf{m}\cdot\mathbf{m} - r^2 = 0$.

O **Ray Tracing** de Whitted [9] estende o Ray Casting com raios secundários: do ponto atingido, um *raio de sombra* vai até a luz, e se algum objeto o interceptar resta apenas a componente ambiente; em superfícies espelhadas, um *raio refletido* $\mathbf{R} = \mathbf{D} - 2(\mathbf{D}\cdot\mathbf{N})\mathbf{N}$ é traçado recursivamente e sua cor é misturada à local com o coeficiente de reflexão k_r. Sombras e reflexões, que no pipeline de rasterização exigem técnicas à parte, surgem naturalmente.

3 Metodologia e Arquitetura da Aplicação

A aplicação foi escrita em Python 3 com pygame-ce 2.5.2 (janela e entrada), PyOpenGL 3.1.7 e NumPy 2.1.3, com OpenGL 3.3 e GLSL 3.30. A Tabela 1 resume os módulos e a origem de cada funcionalidade. O visualizador abre o mini Ray Tracer pela tecla `Y`; os dois também funcionam separadamente.

Tabela 1: Módulos da aplicação e origem de cada funcionalidade.

Módulo	**Conteúdo**	**Origem**
`leitor_vtk.py`, `cores.py`	Leitor VTK e mapas de cores jet e viridis (TP1)	própria
`geometria.py`	Tubos (cilindros e esferas), normais de vértice e de face, orientação das faces	própria
`transformacoes.py`	Matrizes de modelo, câmera e projeções (TP1)	própria
`shaders.py`	Blinn–Phong, Flat, Gouraud, Phong, transparência, textura e bilinear em GLSL	própria
`visualizador.py`	Culling por $\mathbf{N} \cdot \mathbf{V}$, linearização do Z-Buffer, Painter's Algorithm, filtro por raio	própria
`textura.py`	Textura gerada, coordenadas (u, v), NEAREST e bilinear em NumPy	própria
`raycasting.py`	Raio pelo pixel, interseção raio–cilindro e raio–esfera	própria
`raytracer.py`	Mini Ray Tracer com raios primários, de sombra e refletidos	própria
`metricas.py`	Grafo, BFS, gerações, comprimento, volume, bifurcações	própria
—	Compilação de shaders, rasterização, interpolação de atributos, teste de profundidade, culling "OpenGL", blending, amostragem de textura, MSAA	OpenGL

Geometria. Cada segmento vira um cilindro de 12 lados (24 triângulos) e cada ponto da árvore, uma esfera de 8×4 setores que fecha as junções, com o raio do vaso mais grosso que passa por ele. Os triângulos de área nula nos polos das esferas são removidos, restando 48 por esfera; o modelo Nterm512 tem 73 704 triângulos. A espessura desenhada é $r \times 10^{-3}\times$ exagero (padrão $2\times$). Depois de gerados, todos os triângulos são verificados: se $(\mathbf{v}_1 - \mathbf{v}_0) \times (\mathbf{v}_2 - \mathbf{v}_0)$ discorda da média das normais dos vértices, $\mathbf{v}_1$ e $\mathbf{v}_2$ são trocados, garantindo a orientação anti-horária exigida pelo culling do OpenGL. Para o Flat, uma normal de face por triângulo é repetida nos três vértices e enviada no lugar da normal de vértice.

Pipeline programável. Os vértices, cores, normais de vértice e normais de face ficam em VBOs, enviados uma única vez. As matrizes P e VM, calculadas pelas funções do TP1, e a matriz das normais $(VM_{3\times3}^{-1})^T$ vão como *uniforms*, assim como a luz, cuja posição no mundo é levada ao espaço da câmera por V. A iluminação é calculada no espaço da câmera, onde o observador está na origem e $\mathbf{V} = -\mathbf{p}$. No Flat, o atributo de saída usa o qualificador `flat`, que desliga a interpolação; no Gouraud, a função de iluminação é chamada no vertex shader; no Phong, no fragment shader, com a normal interpolada renormalizada. Os coeficientes são $k_a = 0{,}25$, $k_d = 0{,}75$, $k_s = 0{,}6$ e $\alpha = 40$; a especular é anulada quando $\mathbf{N} \cdot \mathbf{L} \leq 0$, para não haver brilho no lado escuro. As teclas `K` e `L` ligam as componentes uma a uma e movem a luz, e um uniform pinta de vermelho os fragmentos com `gl_FrontFacing` falso, revelando faces de trás.

Visibilidade. No modo "próprio", a cada quadro as normais de face e os centroides dos triângulos são levados ao espaço da câmera em NumPy, o critério $\mathbf{N} \cdot \mathbf{V} > 0$ seleciona os triângulos de frente e só os seus índices são enviados num *element buffer* e desenhados com `glDrawElements`. No modo "OpenGL", são usados `glFrontFace(GL_CCW)` e `glCullFace(GL_BACK)`. O Z-Buffer pode ser desligado; nesse caso vale o último triângulo desenhado, na ordem do VBO. Para visualizá-lo, o buffer de profundidade é lido com `glReadPixels`, linearizado pela Equação (2) com $n = 0{,}01$ e $f = 100$ e normalizado entre a menor e a maior distância visíveis, com o mais próximo em tom mais escuro.

Transparência. A opacidade dos vasos é 1,0, 0,6 ou 0,3. Com o Painter's Algorithm ativo, os triângulos são ordenados a cada quadro pela coordenada z do centroide no espaço da câmera (mais negativo = mais distante). O plano opaco é desenhado antes; durante o desenho dos transparentes, o teste de profundidade continua ativo, para que objetos opacos os escondam, mas a escrita no Z-Buffer é desligada (`glDepthMask(GL_FALSE)`), para que um vaso transparente não impeça a composição dos que estão atrás. O modo "sem ordenação" desenha na ordem do VBO, para comparação.

Textura. A textura é gerada em NumPy: uma imagem de 32×32 texels, propositalmente pequena, com um quadriculado 4×4 e o contorno circular do domínio de perfusão. Ela é aplicada a um plano

de referência sob a árvore, com (u, v) de $(0, 0)$ a $(1, 1)$ nos cantos, e iluminada pela mesma luz. Há quatro modos: NEAREST e bilinear pelo OpenGL (`GL_NEAREST`, `GL_LINEAR`); bilinear própria, que lê os quatro texels com `texelFetch` (que não filtra) e aplica a Equação (5); e a exibição das próprias coordenadas, com u no canal vermelho e v no verde.

Ray Casting. No clique direito, o pixel vira coordenadas normalizadas (x, y); os pontos $(x, y, -1)$ e $(x, y, 1)$ do cubo NDC, levados ao mundo por $(PV)^{-1}$ e divididos por w, dão a origem e a direção do raio, o que serve às duas projeções. O raio é levado ao espaço do modelo por M^{-1} (direção com $w = 0$), onde o parâmetro t é preservado, e testado, de forma vetorizada, contra todos os vasos visíveis no filtro, cada um modelado como cilindro (Eq. 6) mais duas esferas, a mesma forma dos tubos. O vaso de menor t é pintado de magenta, cor ausente dos dois mapas, e suas informações são exibidas.

Mini Ray Tracer. Escrito inteiramente em NumPy, sem OpenGL, traça todos os raios de uma vez como vetores. A cena é uma bifurcação CCO simplificada (três cilindros com raios decrescentes e esferas nas junções, coloridos pelo mapa jet), uma esfera espelhada ($k_r = 0{,}8$), uma esfera fosca e um piso quadriculado levemente refletor ($k_r = 0{,}25$), iluminados por uma luz pontual. Os raios secundários partem de $P + 10^{-3}\mathbf{N}$ para evitar que a superfície intercepte o próprio raio (*shadow acne*), e as reflexões vão até a profundidade 3. Os modos 1, 2 e 3 correspondem a Ray Casting, Ray Casting com sombras e Ray Tracing completo.

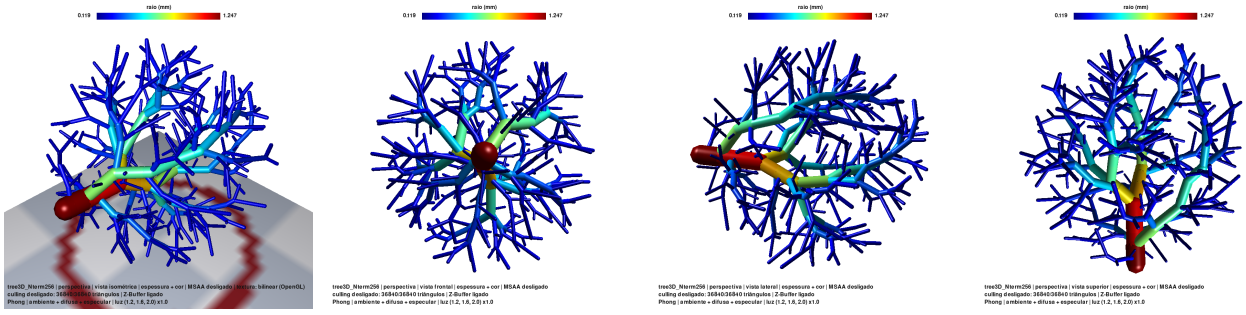
Recurso adicional: métricas e filtro por raio. A árvore é tratada como grafo: pontos de grau 1 são a raiz ou terminais e pontos de grau 3 são bifurcações. A raiz é a extremidade cujo segmento tem maior raio; uma busca em largura a partir dela orienta os segmentos de pai para filho e dá a *geração* de cada vaso (número de bifurcações até a raiz). São calculados o comprimento e o volume $\pi r^2 \ell$ de cada vaso e, em cada bifurcação, a razão $r_p^3/(r_{f_1}^3 + r_{f_2}^3)$, que vale 1 se a lei de Murray [10] for respeitada. O modo intervalo (tecla `I`) ajusta o raio mínimo e máximo exibidos; os vasos fora do intervalo são retirados da lista de triângulos e da seleção, e o painel (tecla `J`) mostra as métricas da árvore e do subconjunto filtrado.

4 Resultados

Todas as figuras foram geradas pelo script `gerar_figuras.py`, que reproduz as configurações automaticamente. O rodapé de cada captura informa os modos ativos e, quando há culling, quantos triângulos foram desenhados.

4.1 Geometria 3D e observador

A Figura 1 mostra o modelo Nterm256 em tubos sobre o plano de referência e em três vistas do observador. A Tabela 4 resume os modelos 3D; os números de triângulos mostram o crescimento linear da malha com o número de vasos.

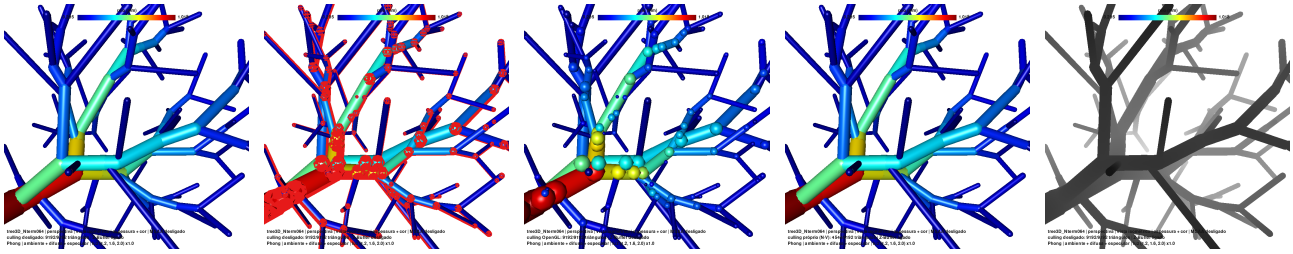


(a) isométrica (b) frontal (c) lateral (d) superior

Figura 1: Modelo 3D Nterm256 representado por tubos com espessura e cor proporcionais ao raio, iluminados por Phong.

4.2 Visibilidade

A Figura 2 usa o destaque das faces de trás em vermelho para separar os efeitos do Z-Buffer e do culling no modelo Nterm064. Com o Z-Buffer ligado (a), nenhuma face vermelha aparece, pois toda face de trás está atrás de uma face de frente do mesmo tubo. Sem Z-Buffer e sem culling (b), vale a ordem de desenho e as faces de trás pintadas por último aparecem por cima, em especial nas esferas das junções. Com o culling do OpenGL e sem Z-Buffer (c), o vermelho desaparece, mas a imagem continua errada: as esferas, desenhadas depois dos cilindros, cobrem os vasos, e vasos distantes aparecem sobre vasos próximos. Por fim, com o Z-Buffer e o culling próprio (d), a imagem é idêntica a (a), mas apenas 4 546 dos 9 192 triângulos (49,5%) foram enviados ao rasterizador. O culling é, portanto, uma otimização, e não um substituto do Z-Buffer. A Figura 2(e) mostra o conteúdo linearizado do Z-Buffer: o tronco, próximo à câmera, é mais escuro, e os ramos ao fundo, mais claros.

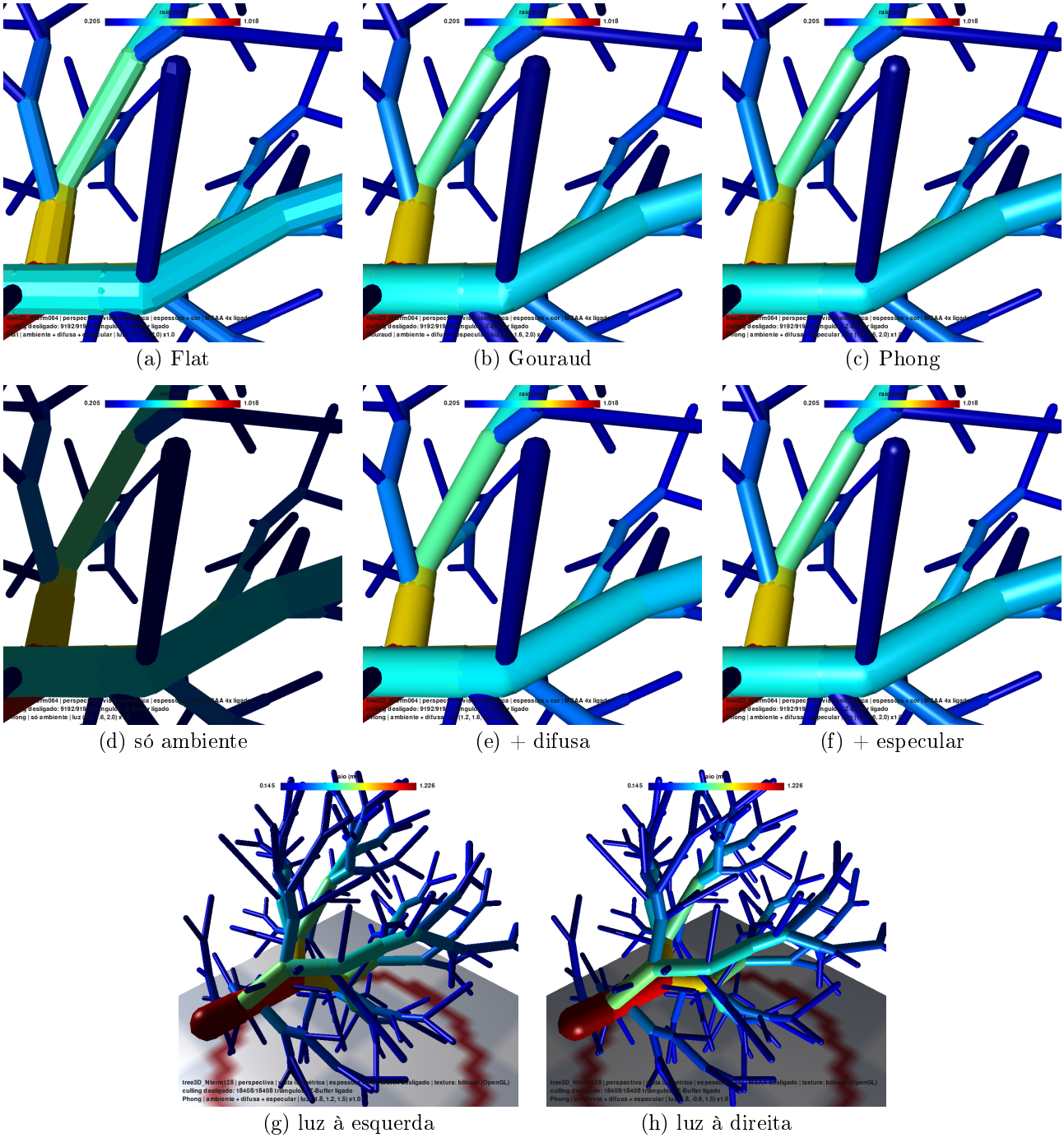


(a) Z-Buffer (b) sem Z, sem culling (c) sem Z, culling GL (d) Z + culling próprio (e) profundidade

Figura 2: Visibilidade no modelo Nterm064 com as faces de trás destacadas em vermelho. (d) é idêntica a (a) desenhando 49,5% dos triângulos; (e) mostra a profundidade linearizada (perto = escuro).

4.3 Iluminação e sombreamento

Na Figura 3, as linhas superiores comparam os três sombreamentos e as componentes do modelo de iluminação, de perto, para que as facetas fiquem visíveis. No Flat (a), cada um dos 12 lados dos cilindros tem uma cor uniforme e os tubos parecem prismas. No Gouraud (b), a transição entre os lados é suave. No Phong (c), o resultado é semelhante ao Gouraud nos vasos finos, em que cada triângulo cobre poucos pixels, e a diferença aparece no brilho especular dos vasos grossos: como os triângulos dos cilindros são longos e estreitos, o Gouraud interpola o brilho ao longo de todo o comprimento do triângulo, enquanto o Phong o calcula em cada pixel. Só com a componente ambiente (d), todos os pontos de um vaso têm a mesma cor e a forma cilíndrica desaparece; a difusa (e) revela o volume e a especular (f) acrescenta o brilho que indica a posição da luz. Na última linha, a mesma árvore iluminada pela esquerda (g) e pela direita (h) mostra como a luz pontual móvel desloca as regiões claras e escuras.



(a) Flat (b) Gouraud (c) Phong

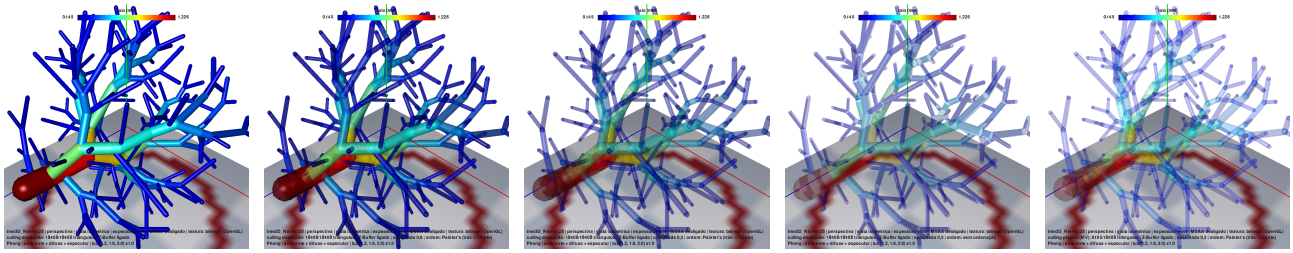
(d) só ambiente (e) + difusa (f) + especular

(g) luz à esquerda (h) luz à direita

Figura 3: Sombreamento (a–c), componentes de Blinn–Phong (d–f) e posição da luz pontual (g, h). Modelos Nterm064 (a–f) e Nterm128 (g, h).

4.4 Transparência

A Figura 4 mostra o modelo Nterm128 com opacidades decrescentes. Com $\alpha = 0{,}3$, os vasos do interior da árvore e o contorno do domínio no plano tornam-se visíveis através dos ramos da frente, o que é útil para entender a estrutura em profundidade. Comparando (c) e (d), a ordenação muda a cor nas regiões em que vasos se sobrepõem: sem ordenação, um vaso desenhado depois de outro que está à sua frente é composto por cima, como se estivesse na frente. A diferença é sutil nesta árvore porque os vasos são finos e se sobrepõem em áreas pequenas. Com o culling próprio (e), cada tubo contribui com uma única camada, a face da frente, em vez de duas; os vasos ficam mais claros e uniformes, e o número de triângulos a ordenar cai de 18 408 para 9 105.



(a) opaco (b) $\alpha = 0{,}6$ (c) $\alpha = 0{,}3$, Painter (d) $\alpha = 0{,}3$, sem ordem (e) (c) + culling

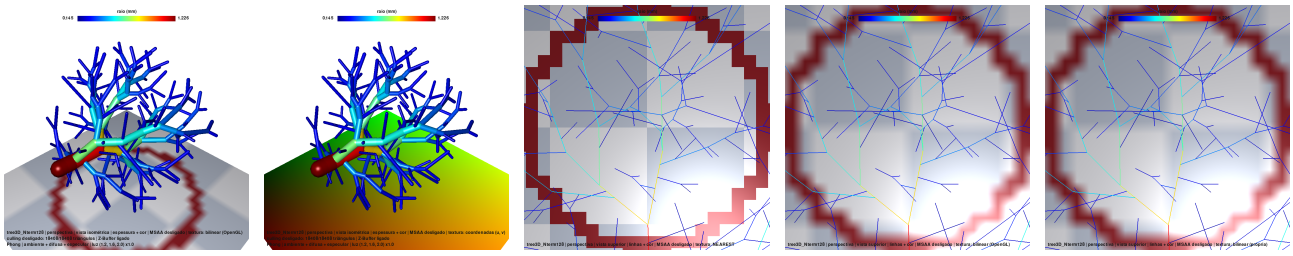
Figura 4: Transparência com alpha blending no modelo Nterm128. (c) e (d) diferem apenas na ordem de composição dos triângulos.

4.5 Textura

A Tabela 2 traz o exemplo numérico da filtragem no ponto $(u, v) = (0{,}245;\ 0{,}50)$, na fronteira entre quatro células do quadriculado. Com $n = 32$, $t = (7{,}34;\ 15{,}50)$ e os vizinhos são os texels $i \in \{7, 8\}$, $j \in \{15, 16\}$. O NEAREST devolve a cor clara de um único texel; a bilinear, com $f_y = 0{,}5$, devolve exatamente a média das duas cores, independentemente de f_x, porque os texels diagonais são iguais. A implementação em NumPy e o shader com `texelFetch` usam a mesma convenção de centros dos texels, e as imagens da bilinear própria e da bilinear do OpenGL na Figura 5 são visualmente indistinguíveis. Na vista superior ampliada, o NEAREST (c) mostra o contorno do domínio como uma escada de blocos, enquanto as duas bilineares (d, e) o suavizam.

Tabela 2: Filtragem no ponto $(u, v) = (0{,}245;\ 0{,}50)$ da textura 32×32: $f_x = 0{,}34$, $f_y = 0{,}50$.

Texel	**Posição** (i, j)	**Cor RGB**
$\mathbf{c}_{00}$	$(7, 15)$	$(190, 198, 214)$
$\mathbf{c}_{10}$	$(8, 15)$	$(235, 235, 240)$
$\mathbf{c}_{01}$	$(7, 16)$	$(235, 235, 240)$
$\mathbf{c}_{11}$	$(8, 16)$	$(190, 198, 214)$
NEAREST	$(\lfloor 0{,}245 \cdot 32 \rfloor, \lfloor 0{,}5 \cdot 32 \rfloor) = (7, 16)$	$(235, 235, 240)$
bilinear	Equação (5)	$(212{,}5;\ 216{,}5;\ 227{,}0)$



(a) plano (b) coordenadas (u, v) (c) NEAREST (d) bilinear OpenGL (e) bilinear própria

Figura 5: Plano de referência texturizado. (b) u no canal vermelho e v no verde. (c–e) vista superior ampliada com a árvore em linhas.

4.6 Ray Casting

No exemplo numérico do `raycasting.py`, o cilindro de $(0, 0, 0)$ a $(0, 0, 10)$, com $r = 1$, é atingido pelo raio $\mathbf{O} = (5, 0, 5)$, $\mathbf{D} = (-1, 0, 0)$ em $t = 4$, no ponto $(1, 0, 5)$, como esperado. Deslocando o raio para $z = 12$, a projeção no eixo sai do intervalo $[0, 10]$ e não há interseção lateral; num vaso, essa região é coberta pela esfera da extremidade. Na Figura 6, o clique sobre um vaso do Nterm128 selecionou o

vaso 9 (geração 2, raio 0,893 mm, comprimento 6,26 mm, $t = 0{,}0202$), destacado em magenta. Como o vaso de menor t é escolhido, um clique onde dois vasos se sobrepõem seleciona sempre o da frente.

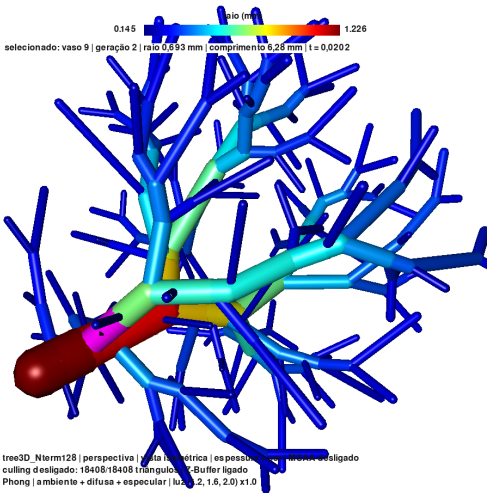
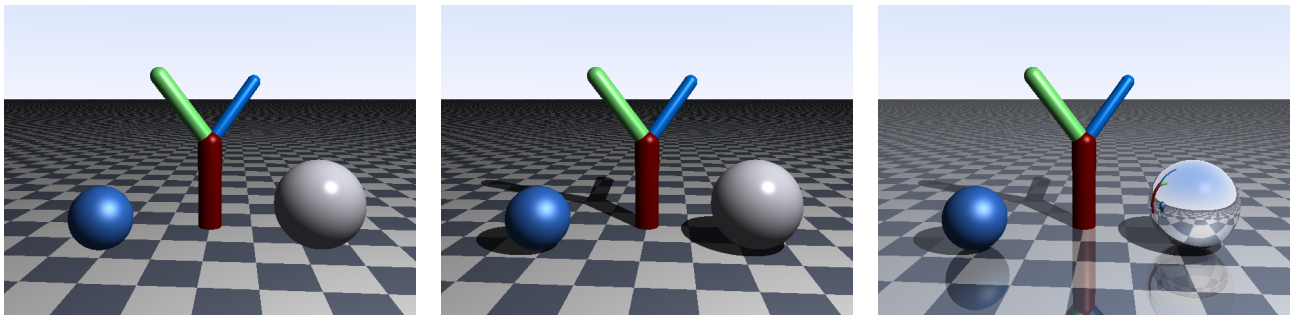


Figura 6: Seleção por Ray Casting: o vaso atingido primeiro pelo raio do pixel clicado aparece em magenta, e suas informações, no topo da tela.

4.7 Ray Tracing

A Figura 7 mostra os três modos do mini Ray Tracer em 640×480 e a Tabela 3, o número de raios de cada tipo. No Ray Casting (a), os objetos estão iluminados e com brilho especular, mas parecem flutuar sobre o piso. Os raios de sombra (b) projetam a bifurcação e as esferas no piso, o que ancora os objetos e revela suas posições relativas. As reflexões (c) fazem a esfera espelhada refletir a bifurcação, o céu e o piso, e o piso refletir os objetos. Os raios de sombra são lançados apenas dos pixels que atingem algum objeto (os que vão para o céu não geram raios secundários), e no modo completo também dos pontos atingidos pelos raios refletidos, o que explica o aumento de 216 228 para 260 377 raios de sombra.



(a) Ray Casting (b) + sombras (c) Ray Tracing completo

Figura 7: Mini Ray Tracer numa cena inspirada numa bifurcação CCO: (a) só raios primários; (b) raios de sombra; (c) raios de sombra e refletidos (profundidade 3).

Tabela 3: Raios traçados por modo (640×480 pixels). Tempos medidos em NumPy, na máquina de desenvolvimento.

Modo	Primários	Sombra	Refletidos	Total	Tempo (s)
1. Ray Casting	307 200	0	0	307 200	0,76
2. + sombras	307 200	216 228	0	523 428	0,75
3. Ray Tracing completo	307 200	260 377	225 348	792 925	1,36

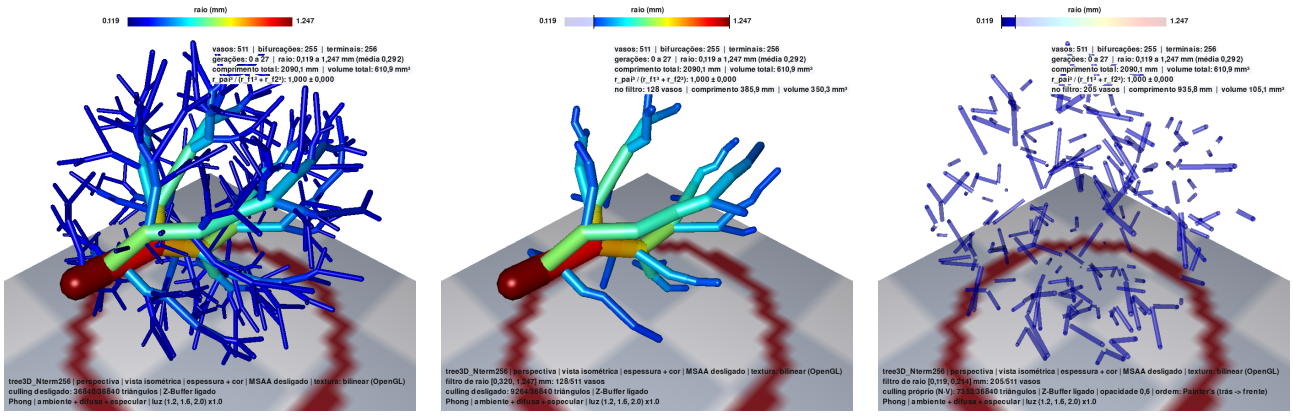
4.8 Recurso adicional: métricas e filtro por raio

A Tabela 4 traz as métricas dos modelos 3D. Em todos, há $N_{\text{term}} - 1$ bifurcações e N_{term} terminais, como numa árvore binária, e a geração máxima cresce com o tamanho, de 8 a 33. A razão $r_p^3/(r_{f_1}^3 + r_{f_2}^3)$ é exatamente 1 em todas as bifurcações, o que confirma que os modelos foram gerados com a lei de Murray com expoente 3. O comprimento total cresce de forma sublinear, de 380 a 3 168 mm quando N_{term} é multiplicado por 32, enquanto o volume cresce bem menos, pois os vasos novos são cada vez mais finos.

Tabela 4: Métricas dos modelos 3D calculadas por `metricas.py` e número de triângulos dos tubos.

Modelo	Bifurc.	Term.	Ger. máx.	Compr. (mm)	Vol. (mm³)	$r_p^3/\sum r_f^3$	Triângulos
Nterm016	15	16	8	379,6	132,1	1,000	2 280
Nterm032	31	32	11	596,9	217,8	1,000	4 584
Nterm064	63	64	15	916,9	352,8	1,000	9 192
Nterm128	127	128	19	1 378,1	547,7	1,000	18 408
Nterm256	255	256	27	2 090,1	610,9	1,000	36 840
Nterm512	511	512	33	3 168,0	671,5	1,000	73 704

A Figura 8 aplica o filtro ao Nterm256. Os 128 vasos mais grossos (raio $\geq 0{,}320$ mm, o quartil superior) formam o esqueleto da árvore (b): são 25% dos vasos e 18,5% do comprimento, mas 57,3% do volume (350,3 de 610,9 mm³). Os 205 vasos mais finos (c), ao contrário, somam 44,8% do comprimento e apenas 17,2% do volume, e aparecem espalhados por todo o domínio, como esperado para os ramos terminais que fazem a perfusão. Em (c), a transparência com culling próprio deixa ver o contorno do domínio sob os vasos.



(a) árvore completa (b) 25% mais grossos (c) 40% mais finos

Figura 8: Filtro por intervalo de raio no modelo Nterm256, com o painel de métricas da árvore e do subconjunto filtrado.

5 Discussão

Culling e Z-Buffer. Os experimentos da Figura 2 mostram papéis distintos. O culling remove faces de trás *de um mesmo objeto fechado* e reduz o trabalho do rasterizador pela metade, mas não decide qual de dois vasos está na frente; o Z-Buffer decide isso pixel a pixel, independentemente da ordem de desenho, ao custo de memória e de um teste por fragmento. O culling próprio e o do OpenGL produzem o mesmo resultado, por critérios diferentes: o próprio usa a geometria (normal e posição) antes da projeção; o do OpenGL, a ordem dos vértices depois dela, e por isso depende da orientação consistente das faces, garantida em `geometria.py`. Em compensação, o culling próprio custa uma operação vetorial em NumPy e o reenvio do *element buffer* a cada quadro, enquanto o do OpenGL é

feito em hardware sem custo perceptível. O valor do próprio é didático e de controle: permite contar os triângulos descartados, informação que o OpenGL não fornece.

Sombreamento. O custo cresce do Flat (uma avaliação por triângulo) ao Phong (uma por pixel). Com 12 lados por cilindro, a diferença entre Gouraud e Phong é pequena nos vasos finos e visível no brilho dos grossos. Para esta aplicação, o Phong é a melhor escolha: o custo extra é absorvido pela GPU e o brilho especular, que indica a curvatura dos tubos, fica correto. O Flat é útil apenas para evidenciar a malha.

Painter's Algorithm. A ordenação pelo centroide é simples, mas falha onde triângulos se interpenetram, o que ocorre nas junções, em que esferas e cilindros se cruzam. Além disso, a ordenação de $O(T \log T)$ é refeita a cada quadro em Python; com o Nterm512 (73 704 triângulos) a rotação fica lenta na renderização por software. Técnicas como *depth peeling* ou transparência independente de ordem (OIT) resolveriam os casos de interpenetração; uma otimização simples seria reordenar apenas quando a câmera muda.

Rasterização e Ray Tracing. A rasterização com Z-Buffer desenha dezenas de milhares de triângulos em tempo interativo, mas sombras e reflexões exigiriam passes extras (*shadow mapping*, mapas de ambiente). No Ray Tracing elas surgem do próprio algoritmo, e as superfícies são exatas (cilindros e esferas analíticos, sem facetas), mas o custo é proporcional a pixels × objetos × raios por pixel: a cena simplificada, com 9 objetos, leva cerca de 1,4 s por quadro em NumPy. Aplicar o mesmo traçador à árvore completa exigiria uma estrutura de aceleração, como uma hierarquia de volumes envolventes (BVH), que reduziria o teste de $O(N)$ para cerca de $O(\log N)$ objetos por raio. O mesmo vale para a seleção por Ray Casting, que hoje testa todos os vasos, mas, por ser um único raio vetorizado, leva cerca de 0,6 ms mesmo no Nterm512 (1 023 vasos).

Limitações. O mini Ray Tracer usa uma cena simplificada, sem antialiasing e com uma única luz. A transparência por Painter's Algorithm não trata interpenetrações. A janela tem tamanho fixo. No ambiente de desenvolvimento (WSL com Mesa llvmpipe), o modelo Nterm512 em tubos com transparência ordenada fica com taxa de quadros baixa.

6 Conclusão

O trabalho cumpriu os objetivos do TP2: o visualizador do TP1 passou a usar um pipeline programável com shaders próprios, em que as equações de iluminação e os três modelos de sombreamento são código da autora; a visibilidade foi estudada com Back-Face Culling próprio e do OpenGL e com a visualização do Z-Buffer; a transparência foi implementada com Painter's Algorithm; a textura, com filtragem bilinear própria validada por um exemplo numérico; e as técnicas baseadas em raios foram usadas tanto na interação (seleção de vasos por Ray Casting) quanto na síntese de imagens (mini Ray Tracer com sombras e reflexões). O recurso adicional de métricas e filtro por raio mostrou, por exemplo, que um quarto dos vasos concentra mais da metade do volume da árvore. Como trabalhos futuros, destacam-se aplicar o Ray Tracer à árvore completa com uma BVH, adotar uma técnica de transparência independente de ordem e acrescentar sombras ao visualizador por *shadow mapping*.

Referências

[1] SCHREINER, W.; BUXBAUM, P. F. Computer-optimization of vascular trees. **IEEE Transactions on Biomedical Engineering**, v. 40, n. 5, p. 482–491, 1993.

[2] FOLEY, J. D.; VAN DAM, A.; FEINER, S. K.; HUGHES, J. F. **Computer Graphics: Principles and Practice**. 2. ed. Reading: Addison-Wesley, 1990.

[3] CATMULL, E. **A Subdivision Algorithm for Computer Display of Curved Surfaces**. Tese (Doutorado) – University of Utah, Salt Lake City, 1974.

[4] PHONG, B. T. Illumination for computer generated pictures. **Communications of the ACM**, v. 18, n. 6, p. 311–317, 1975.

[5] BLINN, J. F. Models of light reflection for computer synthesized pictures. **ACM SIGGRAPH Computer Graphics**, v. 11, n. 2, p. 192–198, 1977.

[6] GOURAUD, H. Continuous shading of curved surfaces. **IEEE Transactions on Computers**, v. C-20, n. 6, p. 623–629, 1971.

[7] NEWELL, M. E.; NEWELL, R. G.; SANCHA, T. L. A solution to the hidden surface problem. In: **Proceedings of the ACM Annual Conference**, p. 443–450, 1972.

[8] APPEL, A. Some techniques for shading machine renderings of solids. In: **Proceedings of the AFIPS Spring Joint Computer Conference**, p. 37–45, 1968.

[9] WHITTED, T. An improved illumination model for shaded display. **Communications of the ACM**, v. 23, n. 6, p. 343–349, 1980.

[10] MURRAY, C. D. The physiological principle of minimum work: I. The vascular system and the cost of blood volume. **Proceedings of the National Academy of Sciences**, v. 12, n. 3, p. 207–214, 1926.